A RAG-Powered Chatbot for CyberArk API Documentation

DISSERTATION

Submitted in partial fulfilment of the requirements of the

Degree: MTech in Artificial Intelligence and Machine Learning

By

Saya Venkata Thrivedi

2023AA05203

Under the supervision of

Sasikiran Varikunta

(Staff Software Engineer, CyberArk)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE

Pilani (Rajasthan) INDIA

(June 2025)

Abstract:

This dissertation investigates the design and partial implementation of a Retrieval-Augmented Generation (RAG)-based chatbot aimed at improving access to complex API documentation. The system is intended to assist developers in querying large, semi-structured specifications by providing context-aware, accurate, and conversational responses. The focus is on processing publicly available OpenAPI specification files published by an organization to describe its REST APIs in a structured format.

Work completed to date includes systematic data collection, parsing and transforming OpenAPI files into retrievable text chunks, and enriching these with metadata to enhance semantic search. Multiple embedding models and retrieval strategies were compared to identify the most effective combination for retrieving relevant documentation sections. A console-based prototype has been developed to demonstrate the integrated retrieval and generation pipeline, supporting realistic technical queries with grounded responses.

Future work will focus on refining the search results to improve answer fluency, developing an interactive user interface, expanding domain coverage, and conducting formal evaluations. The project aims to contribute a scalable framework for intelligent, conversational access to technical documentation in software development contexts.

Table of contents:

Abstract..... 2
Table of contents..... 3
List of Abbreviations..... 5
List of Tables..... 6
List of Figures..... 7
Methodology..... 8
Chapter 1. System Architecture..... 8
1.1 Data Collection..... 8
1.2 Data Processing..... 9
1.3 Data Enrichment..... 9
1.4 Embedding Generation..... 9
1.5 Retrieval and Generation..... 10
1.6 Design Considerations and Modularity..... 10
Chapter 2. Data Collection and Processing..... 11
2.1 Data Collection..... 11
2.2 Data Processing and Format Conversion..... 11
2.3 Specification Parsing and Chunking..... 11
2.4 RAG-Optimized Formatting..... 12
2.5 Data Enrichment..... 12
2.5.1 Motivation for Enrichment..... 12
2.5.2 Automated Enrichment Pipeline..... 13
2.6 Structured Markdown Generation..... 13
2.7 Advantages of the Enrichment Approach..... 14
Chapter 3. Embedding Comparison and Creation..... 15
3.1 Motivation and Goals..... 15
3.2 Experimental Setup..... 15
3.2.1 Evaluation Process..... 15
3.3 Results and Interpretation..... 16
3.3.1 Embedding Time Comparison..... 16
3.3.2 Query Time Analysis..... 17
3.3.3 Retrieval Quality (NDCG)..... 17
3.3.4 Embedding Dimension Considerations..... 18
3.3.5 API Keyword Coverage..... 18
3.3.6 Composite Score and Model Recommendation..... 19
3.4 Saving and Deployment..... 19
Chapter 4. Comparative Analysis and Selection of Retrieval Strategies..... 20
4.1. Introduction..... 20
4.2 Overview of Evaluated Retrieval Methods..... 20
4.2.1 Basic Dense Vector Retrieval..... 20
4.2.2 Maximum Marginal Relevance (MMR)..... 20
4.2.3 Similarity Score Thresholding..... 21
4.2.4 MultiQuery Retrieval..... 21
4.2.5 Custom MultiQuery Retrieval..... 21

4.2.6 BM25-Based Keyword Retrieval..... 21
4.2.7 Ensemble Retrieval..... 22
4.3 Comparative Analysis..... 22
4.4. Justification for Selecting the Ensemble Approach..... 22
Chapter 5. QA Chain and Response Generation..... 24
5.1 Introduction..... 24
5.2 Architecture and Tooling..... 24
5.3 QA Chain Design..... 24
5.4 Prompt Engineering..... 24
5.5 Context Formatting..... 25
5.6 Justification for Technique..... 25
5.7 Model Selection and Configuration..... 25
5.8 Rate Limiting and Production Considerations..... 26
5.9 Advantages, Limitations and Mitigation Strategies..... 26
Directions for Future Work..... 28
1. Implementation of Output Parsers..... 28
2. Input and Output Guardrails..... 28
3. Context-Aware and Multi-Turn Dialogue..... 28
4. User Interface Development..... 28
5. Evaluation and Benchmarking..... 29
Bibliography / References..... 30

List of Abbreviations

RAG	:	Retrieval-Augmented Generation
API	:	Application Programming Interface
OpenAPI	:	Open Application Programming Interface
OLLAMA	:	Omni-Layer Learning Language Acquisition Model
YAML	:	YAML Ain't Markup Language
JSON	:	JavaScript Object Notation
SLM	:	Small Language Model
NDCG	:	Normalized Discounted Cumulative Gain
BM25	:	Best Matching 25
CSV	:	Comma-Separated Values
HTTP	:	Hypertext Transfer Protocol
REST	:	Representational State Transfer
FAISS	:	Facebook AI Similarity Search
BGE	:	BAAI General Embedding

List of Tables

Table-1 : Comparison of retrievers..... 22

List of Figures

Figure-1 : System Architecture of the Retrieval-Augmented Generation (RAG)-Based... **8**
Figure-2 : Embedding Generation Time for Each Model..... **16**
Figure-3 : Average Query Time Across Models..... **17**
Figure-4 : Average NDCG Scores for Retrieval Quality..... **17**
Figure-5 : Embedding Dimensions for Each Model..... **18**
Figure-6 : API Keyword Hit Rate Across Models..... **18**
Figure-7 : Composite Model Scores..... **19**
Figure-8 : Sample QnA on console chat..... **26**
Figure-9 : Sample QnA on console chat..... **27**
Figure-10 : Sample QnA on console chat with out of context question..... **27**

Methodology

Chapter 1. System Architecture

The proposed system implements a Retrieval-Augmented Generation (RAG) pipeline specifically designed to facilitate question answering over complex API documentation. The primary objective is to enable developers to query API specifications conversationally and receive accurate, grounded responses that reduce time spent manually searching documentation.

The overall architecture comprises five key stages: Data Collection, Data Processing, Data Enrichment, Embedding Generation, and Retrieval and Generation. Each stage is designed to modularly transform raw documentation into a format suitable for retrieval-augmented question answering. The system's design is visualized in Figure 3.1, which provides a high-level overview of data flow and processing components.

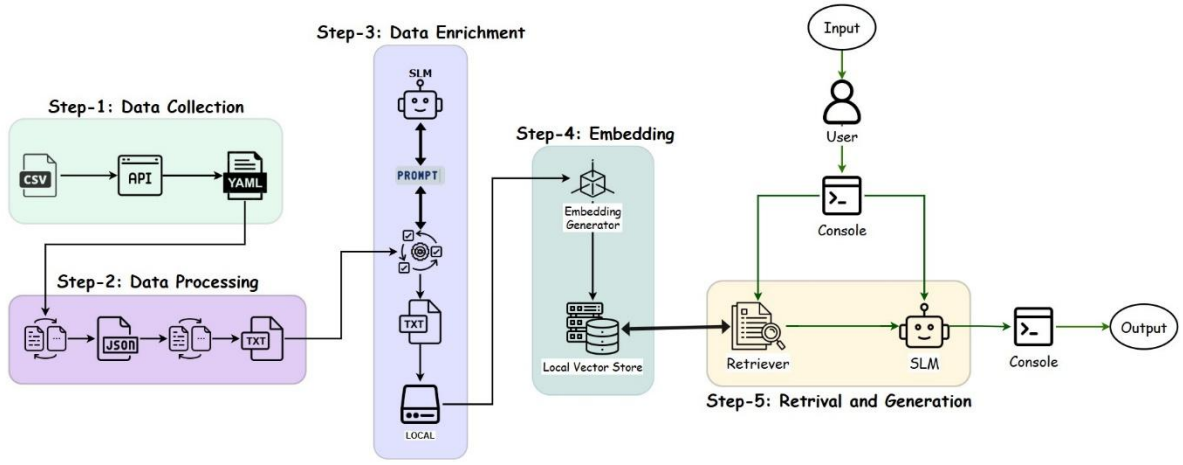


Figure 1: System Architecture of the Retrieval-Augmented Generation (RAG)-Based Chatbot – Image Link

1.1 Data Collection

The system begins with data collection from publicly available API specifications published by the organization. These specifications are typically distributed in OpenAPI YAML format and define RESTful endpoints, parameters, request/response schemas, authentication methods, and additional metadata. The primary sources include developer portals, documentation repositories, or downloadable API specification bundles.

The objective of this stage is to assemble a representative corpus of production-grade API documentation that reflects realistic complexity, heterogeneity, and scale. By collecting raw YAML files directly, the system ensures that no semantic detail is lost prior to parsing, enabling precise representation of endpoint structure and behaviour.

1.2 Data Processing

After collection, the YAML-formatted specifications are programmatically parsed into structured JSON objects to facilitate manipulation. This conversion step is critical because OpenAPI specifications can be deeply nested and require flattening for downstream text-based processing. The processing pipeline includes schema validation, error handling, and normalization to maintain consistency across diverse specifications.

Subsequently, the structured JSON data is segmented into semantically meaningful text chunks. For example, individual chunks may correspond to single endpoints, methods (GET/POST), parameter descriptions, request and response bodies, authentication requirements, or error codes. Chunking is guided by the OpenAPI schema hierarchy to ensure logical boundaries and maximize retrieval relevance.

Processed text chunks are exported to plain text files, forming an initial corpus of documentation units that can be independently retrieved at inference time. This stage enables efficient mapping between user queries and relevant API elements by providing discrete, indexable text segments.

1.3 Data Enrichment

While the raw text chunks capture the original documentation content, they may lack the semantic diversity or clarity needed for high-quality retrieval. To address this, an enrichment step is applied to augment and improve the text data.

Prompt-based generation using a small language model (SLM) is applied to rephrase, expand, or clarify certain text sections. For example, complex parameter descriptions may be paraphrased into simpler language, or additional context may be generated to improve semantic matching during retrieval. This process improves the model's ability to understand diverse user query formulations while preserving technical accuracy.

Enriched text data is stored locally in a structured repository. Maintaining local control over the data ensures security, reproducibility, and ease of iteration during development.

1.4 Embedding Generation

The enriched text corpus is then transformed into dense vector representations using an embedding model. The embedding generator used is selected depending on performance requirements and cost constraints. The goal is to encode the semantic meaning of each text chunk into a high-dimensional vector that enables best similarity-based retrieval.

Embedding generation is a critical step, as retrieval effectiveness relies on the quality of semantic representation. During development, multiple embedding models were compared on criteria such as semantic relevance, latency, and memory footprint. The selected embedding model produces embeddings that are stored in a local vector database.

1.5 Retrieval and Generation

At inference time, the system supports interactive question answering through a console-based interface. The user enters a natural language query, which is first embedded using the same embedding model. The retriever module performs a vector similarity search against the local store to identify the most relevant documentation chunks based on semantic proximity.

The retrieved context is then provided as input to the generation module, which uses a small language model (SLM) to produce a natural language response. The generation process is designed to be retrieval-augmented, ensuring that the answer remains grounded in retrieved documentation and minimizes unsupported hallucinations.

The console interface serves as a lightweight demonstration environment to validate the integrated pipeline. It allows developers to test realistic queries and observe retrieved evidence and generated responses in real time. This prototype approach enables iterative evaluation and refinement of retrieval strategies, embedding models, and generation prompts.

1.6 Design Considerations and Modularity

The architecture is designed to be modular and extensible. Each component data processing, enrichment, embedding, retrieval, generation can be independently updated or replaced without requiring reimplementation of the entire pipeline. This separation of concerns supports experimentation with alternative embedding models, retrievers (e.g., BM25 vs. dense retrieval), or generative backends.

The modular design also enables future scaling, such as deployment via web interfaces, support for multiple API domains, or integration with additional developer tools. By grounding generation in retrieved, organization-authored specifications, the system provides a foundation for trustworthy, transparent, and efficient conversational access to API documentation.

Chapter 2. Data Collection and Processing

The first critical phase in developing the Retrieval-Augmented Generation (RAG)-based chatbot is the acquisition and transformation of high-quality API documentation into a retrieval-friendly corpus. The system targets publicly available OpenAPI specifications that define the organization's REST APIs. These structured documents, typically authored in YAML format, capture detailed information about endpoints, parameters, request/response schemas, authentication requirements, and associated metadata.

2.1 Data Collection

The data collection workflow begins with a curated CSV file that enumerates categories of APIs along with the corresponding URLs where their OpenAPI YAML specifications can be obtained. This approach ensures coverage of multiple API domains, allowing the chatbot to be tested against a realistic, heterogeneous documentation set.

For each entry in the CSV, the system automatically retrieves the OpenAPI YAML specification via HTTP requests. Downloaded specifications are stored locally in a structured directory hierarchy, preserving their original formatting. This local archival guaranteed reproducibility of experiments and supports version control of the raw input data.

A significant design choice in this phase is to retain the original YAML format until explicitly converted, avoiding any lossy preprocessing. By ensuring that specifications are collected in their source-authorized form, semantic and structural fidelity is maintained, supporting downstream parsing without loss of detail.

2.2 Data Processing and Format Conversion

Once collected, the YAML specifications are programmatically parsed and converted to JSON format. This transformation is crucial because JSON representations are more readily manipulated in most programming environments and allow easy traversal of nested hierarchical structures. Each converted JSON file maintains the complete OpenAPI schema, including paths, methods, parameters, security schemes, and reusable components.

The processing pipeline applies schema validation and error handling during conversion to identify and filter out invalid or incomplete specifications. Successfully converted JSON files are stored locally in a dedicated directory structure, serving as the canonical structured input for further analysis.

2.3 Specification Parsing and Chunking

The central goal of data processing is to transform large, complex API specifications into semantically meaningful text units that can be efficiently indexed and retrieved. To achieve this, the system parses each OpenAPI JSON document to extract detailed information about every defined endpoint. This includes:

- Endpoint paths and HTTP methods (e.g., GET, POST)
- Summaries and detailed descriptions
- Tags and categorization metadata

- Parameters, including names, locations, required/optional status, and descriptions
- Request body schemas and example JSON structures
- Response schemas, status codes, and example responses
- Security requirements, such as authentication schemes
- Custom extension fields prefixed with "x-" in the OpenAPI standard

This information is programmatically extracted and structured into discrete text chunks, each representing a single endpoint and its properties in a consistent, human-readable format. The design emphasizes faithful representation of the original documentation while reformatting it to support retrieval by semantic similarity.

2.4 RAG-Optimized Formatting

In addition to direct extraction, the processing pipeline converts the text with several formatting steps aimed at improving its suitability for retrieval-augmented generation. Schema definitions are traversed recursively to list nested properties, data types, and associated descriptions. Sample JSON objects are programmatically generated from schema definitions to provide concrete, example payloads for both requests and responses.

The final output for each endpoint is a structured, detailed text document organized into clearly labelled sections such as ENDPOINT, PATH, METHOD, TAGS, DESCRIPTION, PARAMETERS, REQUEST BODY, RESPONSES, and SECURITY. These consistent structural markers enable fine-grained chunking and improve semantic retrieval precision.

2.5 Data Enrichment

Despite careful data collection and processing, the extracted endpoint documentation remains limited in depth and semantic diversity. Typical OpenAPI specifications, even when cleanly parsed and structured, often contain minimal natural-language context, focusing narrowly on schema details, parameter names, and terse descriptions. Such content is insufficient for effective retrieval-augmented generation (RAG) applications, where semantic similarity search thrives on rich, varied, and human-readable text.

To address this limitation, a dedicated Data Enrichment step was designed and implemented. This step leverages a Small Language Model (SLM) to automatically transform the raw processed endpoint documentation into highly structured, retrieval-optimized text files. The enrichment pipeline systematically expands each endpoint's sparse technical description into a semantically rich, multi-section document designed to support both retrieval and generation tasks.

2.5.1 Motivation for Enrichment

The primary motivation is to bridge the gap between structured API reference data and natural-language information needs. While developers or end-users often ask questions in free-form language (e.g., "How do I delete a user profile?"), the raw documentation might only specify DELETE /user/{id} with no further context. Without enrichment, vector retrieval models struggle to match such queries to the right chunk, resulting in poor RAG performance.

Enrichment addresses this by introducing:

- **Natural-language overviews** summarizing endpoint purpose and usage.
- **Relevant search terms** to improve embedding space coverage.
- **Example user questions** that mimic real query intent.
- **Developer-focused notes** highlighting parameters, request/response schemas, and security considerations.
- **Consistently formatted raw documentation** for full fidelity.

These elements collectively ensure that retrieval is grounded not just in keyword overlap, but in semantic similarity informed by realistic usage contexts.

2.5.2 Automated Enrichment Pipeline

The enrichment pipeline is designed as a batch process operating over the corpus of processed endpoint text files generated in the earlier step. The key stages of this pipeline are

Input Preparation

- All processed text files are collected from a dedicated directory.
- Each file typically contains multiple endpoint sections, separated by well-defined delimiters.
- These sections are split to ensure that enrichment is performed at the *endpoint* level granularity, preserving chunk-level retrieval fidelity.

Prompt Engineering

- A carefully designed enrichment prompt is formulated to guide the SLM in transforming raw endpoint text.
- The prompt includes detailed instructions to produce consistent output with markdown headings, bullet lists, and code blocks where appropriate.
- Importantly, the prompt explicitly instructs the model to use only content from the raw documentation when extracting details, while allowing generic additions for search terms and example questions to boost retrieval effectiveness.

SLM Invocation

- For each endpoint chunk, the system constructs a complete prompt by embedding the raw documentation into the template.
- This prompt is then sent to a locally hosted Small Language Model (SLM), chosen for its balance of performance, controllability, and cost.
- The SLM's deterministic temperature setting ensures consistent, reproducible results across runs.

2.6 Structured Markdown Generation

The model responds with a markdown-formatted document containing the following sections:

- A concise natural-language summary of the endpoint's purpose, HTTP method, path, and security requirements.
- 5–10 keywords or phrases relevant for semantic search.
- Sample natural-language queries likely to match the endpoint.
- Technical details about parameters, request/response schemas, required fields, and error handling.

- Cleaned and consistently formatted reproduction of the original processed text.

This format is designed to optimize both retrieval accuracy (via search terms and user questions) and grounding fidelity (via developer notes and raw documentation). Each enriched endpoint is saved as an individual markdown-formatted text file.

2.7 Advantages of the Enrichment Approach

- **Improved Semantic Retrieval**: By expanding sparse documentation into rich natural language, embedding models can better learn meaningful similarities between user queries and document chunks.
- **Grounded Context**: The structured prompt ensures that enrichment remains faithful to source content, avoiding hallucination while adding helpful context.
- **Consistency Across Endpoints**: Standardized markdown sections enable reliable chunking, indexing, and retrieval.
- **Ease of Maintenance**: New or updated API specifications can be reprocessed and re-enriched automatically, ensuring scalability.

Chapter 3. Embedding Comparison and Creation

After enriching the raw API documentation with structured, retrieval-friendly content, the next critical step in the Retrieval-Augmented Generation (RAG) pipeline is generating high-quality vector embeddings. The goal of this stage is to convert enriched text chunks into numerical representations that preserve semantic similarity for effective retrieval. To ensure optimal performance, multiple embedding models were evaluated and compared across several key metrics, including embedding generation time, query response time, and retrieval quality.

3.1 Motivation and Goals

Embedding quality fundamentally determines the effectiveness of the entire RAG system. If embeddings fail to capture the semantic meaning of user queries and documentation chunks, the retrieval component will return irrelevant or suboptimal results, directly harming the end-user experience. Moreover, practical considerations such as computational cost (embedding time) and serving latency (query time) are also essential, especially for real-time applications.

Given these requirements, the evaluation aimed to identify and embedding model that offers a good balance between:

- Retrieval quality (measured by NDCG score on realistic queries),
- Embedding time (the one-time cost of generating the vector store),
- Query time (the per-request retrieval latency),
- Embedding dimension (affecting storage and compute costs).

3.2 Experimental Setup

The enriched corpus was first split into context-preserving chunks using overlap-aware splitting. Each chunk retained consistent metadata to support traceability.

Three embedding models with varying architectures and dimensionalities were then configured:

- A lightweight Hugging Face model (384 dimensions),
- Nomic Embed (768 dimensions),
- BGE-M3 (1024 dimensions).

These models were selected to represent a spectrum of trade-offs in quality, dimension size, and compute requirements.

3.2.1 Evaluation Process

The evaluation was designed to mimic realistic retrieval scenarios for API documentation chatbots. It followed these main stages:

Vector Store Creation

- Each model generated embeddings for all text chunks and stored them in a FAISS index.
- Embedding generation time was recorded as an important cost metric.

Query Performance Measurement

- A curated set of real-world API-related test queries was used to simulate chatbot questions.
- For each model, the average retrieval latency (query time) was measured.

Relevance Scoring and Ranking Evaluation

- Retrieved chunks were scored for relevance using a heuristic combining keyword overlap and presence of API-specific terms.
- Normalized Discounted Cumulative Gain (NDCG) was computed to measure the quality of ranking, reflecting how well each model prioritized relevant results.

API-Specific Keyword Coverage

- An analysis was conducted to measure each model's ability to retrieve text containing key API-related concepts (e.g., endpoint, parameters, security, authentication).
- This metric assessed domain grounding beyond generic semantic similarity.

Composite Scoring and Recommendation

The models were compared on normalized, weighted metrics to identify the best overall choice for production deployment.

3.3 Results and Interpretation

3.3.1 Embedding Time Comparison

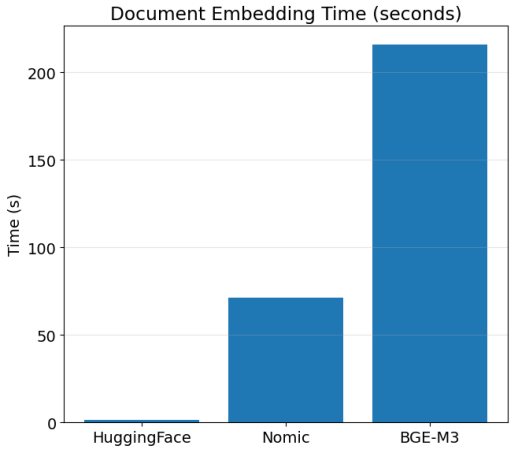


Figure-2: Embedding Generation Time for Each Model

A bar plot comparing embedding generation times showed large variations between models. The Hugging Face model completed embedding in just over ~1.6 seconds, while Nomic Embed required over ~70 seconds, and BGE-M3 exceeded ~215 seconds. This indicates a clear cost advantage for Hugging Face in workflows where embeddings are regenerated frequently (e.g., for dynamic or frequently updated documentation).

3.3.2 Query Time Analysis

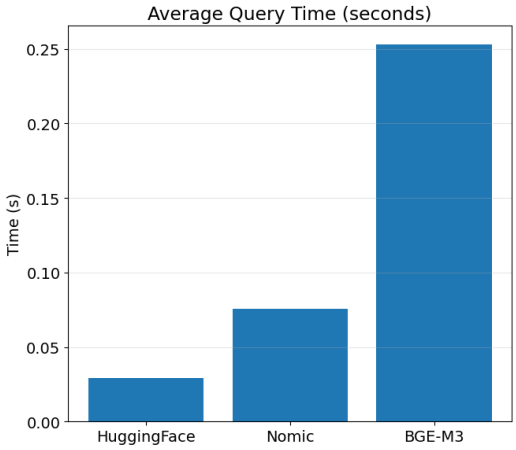


Figure-3: Average Query Time Across Models

The average retrieval latency per query was lowest for the Hugging Face model, with sub-0.05 sec performance. In contrast, Nomic Embed and BGE-M3 showed higher latencies. Low query times are particularly important for conversational AI applications, where users expect near-instant responses.

3.3.3 Retrieval Quality (NDCG)

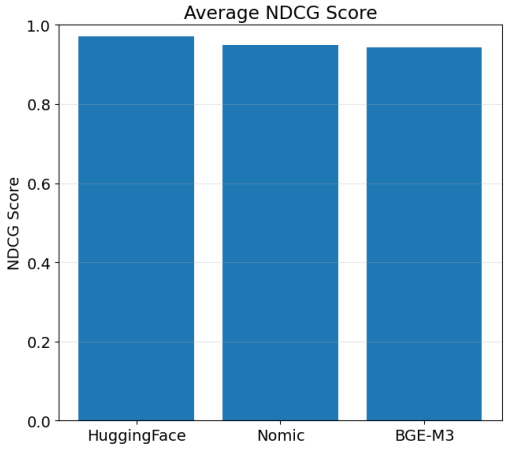


Figure-4: Average NDCG Scores for Retrieval Quality

All models achieved high average NDCG scores, reflecting effective semantic retrieval of API documentation chunks. These results indicate that even the smallest model achieved strong ranking quality, challenging assumptions that larger embeddings automatically yield better retrieval.

3.3.4 Embedding Dimension Considerations

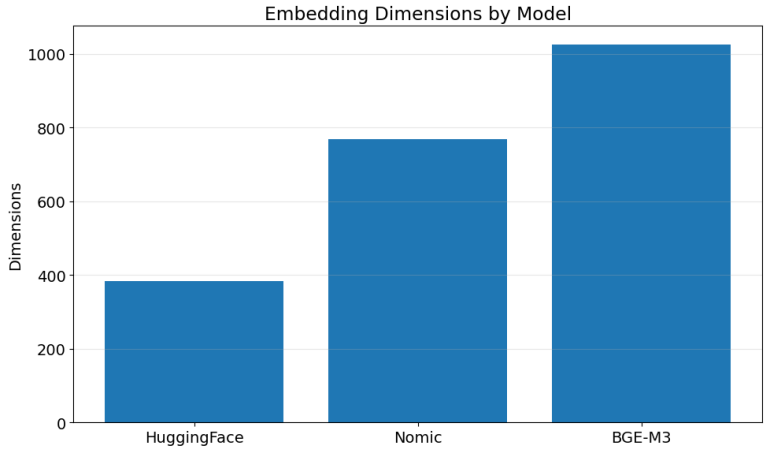


Figure-5: Embedding Dimensions for Each Model

Embedding dimensions directly impact index size, storage requirements, and retrieval latency. Hugging Face's 384-dimensional embeddings offer a significant efficiency advantage, while BGE-M3's 1024-dimensional vectors demand greater resources for comparable quality.

3.3.5 API Keyword Coverage

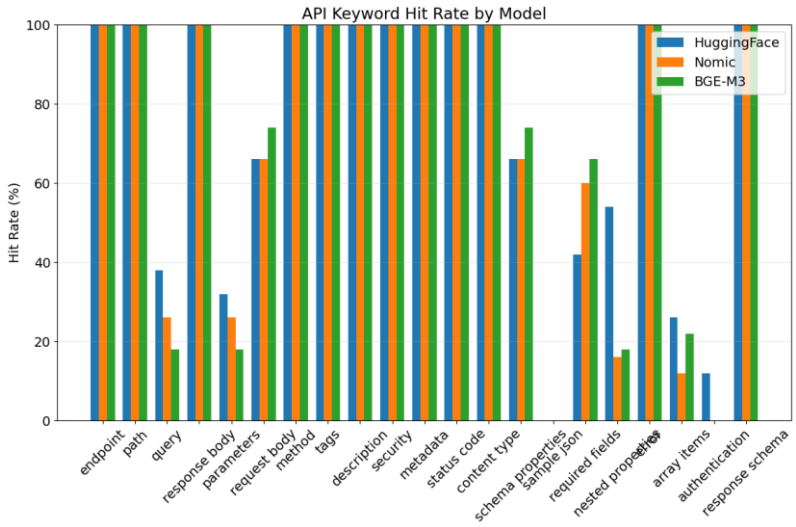


Figure-6: API Keyword Hit Rate Across Models

An additional analysis focused on the proportion of retrieved text chunks containing essential API concepts. Across all models, coverage of terms such as *endpoint*, *path*, *parameters*, and *authentication* was high, confirming that all embeddings maintained strong domain relevance.

However, slight variations were observed, with larger models sometimes surfacing more context-rich passages at the cost of latency.

3.3.6 Composite Score and Model Recommendation

A final composite score was computed for each model, weighting:

- Retrieval quality (NDCG): highest importance,
- Query time: medium importance,
- Embedding time: lower importance (as a one-time cost).

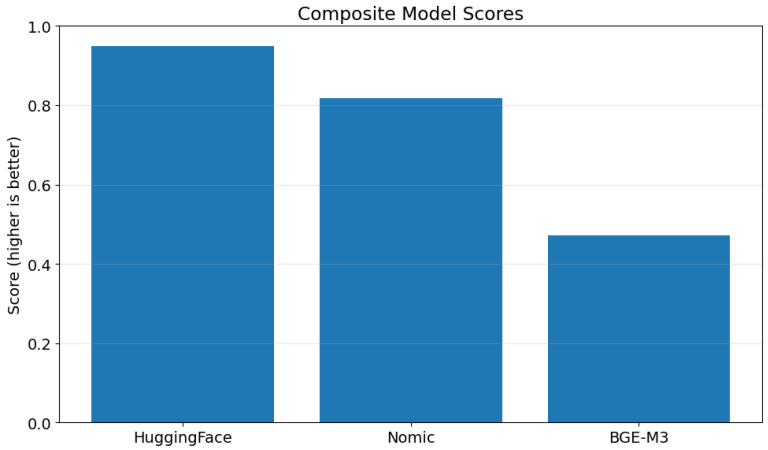


Figure-7: Composite Model Scores

The analysis concluded that the **Hugging Face** model provided the best balance between retrieval accuracy, query responsiveness, and computational cost.

Its combination of minimal embedding time, fastest query time, and highest NDCG score makes it the recommended choice for production deployment in this API-focused RAG chatbot.

3.4 Saving and Deployment

Following selection, the best-performing embedding model was used to create a finalized FAISS vector store of the enriched documentation. This vector index was saved locally for seamless integration into the chatbot's retrieval pipeline.

Chunks were saved with overlap to preserve context, and metadata ensured traceability to the source files. This design enables efficient, high-quality retrieval during chatbot operation while allowing easy re-embedding in the future as documentation evolves.

Chapter 4. Comparative Analysis and Selection of Retrieval Strategies

4.1. Introduction

The retrieval component in a Retrieval-Augmented Generation (RAG) system plays a pivotal role in shaping the relevance and accuracy of generated responses. For the case of an API documentation assistant, the retrieval stage determines which sections of the documentation are supplied to the language model, thereby directly influencing the system's capacity to answer user queries correctly.

Given the structured and technical nature of API specifications containing endpoints, methods, parameters, and schemas it is essential to evaluate retrieval methods systematically. This section presents an analysis of different retrieval strategies, assessing their suitability for this context, and explains the rationale for adopting an ensemble approach in the final system design.

4.2 Overview of Evaluated Retrieval Methods

A range of retrieval strategies were investigated, each employing distinct techniques for ranking and selecting relevant text passages.

4.2.1 Basic Dense Vector Retrieval

This method relies on embedding-based similarity, where both the user query and document chunks are represented as dense vectors. The retrieval is performed by selecting the top-k most similar chunks according to a similarity metric such as cosine similarity.

***Strengths & Limitations**:*

- Conceptual matching beyond exact wording.
- Straightforward implementation.
- May overlook exact phrases critical in API documentation (e.g., paths or parameter names).
- Sensitive to the quality of the embedding model.

4.2.2 Maximum Marginal Relevance (MMR)

Maximum Marginal Relevance introduces a mechanism to balance relevance and diversity in retrieval. By penalizing similarity among selected documents, MMR aims to reduce redundancy and promote coverage of different aspects of a query.

Strengths & Limitations:

- Encourages diverse, non-redundant results.
- Can uncover complementary information.
- May inadvertently exclude highly relevant passages if they are similar to others already selected.
- Slightly increased computational complexity.

4.2.3 Similarity Score Thresholding

This technique retrieves only documents whose similarity score exceeds a predefined threshold. By filtering out low-scoring candidates, it improves precision at the cost of potentially reducing recall.

Strengths & Limitations:

- Reduces retrieval of irrelevant or noisy content.
- Allows fine-grained control over result quality.
- Requires careful threshold tuning.
- Risk of failing to retrieve any documents if the threshold is too strict.

4.2.4 MultiQuery Retrieval

MultiQuery Retrieval expands the user's question into multiple reformulations using a language model. Retrieval is then performed for each variation, and results are aggregated.

Strengths & Limitations:

- Increases coverage by capturing different ways a question might be phrased.
- Particularly useful when user queries are ambiguous or under-specified.
- Computationally more demanding.
- Quality depends on the diversity and relevance of generated variations.

4.2.5 Custom MultiQuery Retrieval

To address domain specificity, a customized MultiQuery Retriever was developed using a prompt tailored to API documentation. The language model was instructed to generate variations that include endpoint names, HTTP methods, and parameter-related terms.

Strengths & Limitations:

- Better alignment with the terminology and structure of API documentation.
- Enhances the likelihood of retrieving domain-relevant content.
- Relies on well-designed prompts.
- Dependent on language model capabilities.

4.2.6 BM25-Based Keyword Retrieval

BM25 is a classical information retrieval algorithm that ranks documents based on exact term matches and term frequency. Unlike embedding-based methods, BM25 excels at identifying precise keyword and phrase matches.

Strengths & Limitations:

- High precision for exact phrases, including endpoint paths and method names.
- Particularly suited for structured text with domain-specific terminology.
- No understanding of semantic similarity.
- Cannot handle paraphrasing or conceptual rewordings.

4.2.7 Ensemble Retrieval

Given the complementary strengths of different retrieval techniques, an ensemble approach was implemented. This method combines the outputs of multiple retrievers using weighted aggregation. For example:

Strengths & Limitations:

- Balances exact keyword matching with semantic similarity.
- Allows tuning to emphasize different retrieval priorities.
- Mitigates weaknesses of individual retrieval strategies.
- Increased complexity in design and maintenance.
- Requires weight tuning to achieve optimal performance.

4.3 Comparative Analysis

An empirical evaluation was conducted by posing representative questions typical of API documentation usage (e.g., "How to delete a policy?"). The methods were compared on metrics such as the number of retrieved documents, relevance of content, and downstream question-answering performance.

Key findings included:

- Basic dense retrieval often missed exact API paths.
- MMR retrieval improved diversity but occasionally reduced relevance.
- Similarity Score Thresholding improved precision but risked excluding valid answers.
- MultiQuery methods significantly improved recall, with the custom version performing better for domain-specific queries.
- BM25 retrieval was highly effective for exact term matching but failed to handle paraphrasing.
- The ensemble approach consistently achieved the best balance, successfully retrieving both precise and semantically related content.

	Retriever Type	Documents Retrieved	Advantages	Limitations
0	Basic Vector	4	Simple, fast	Sensitive to query phrasing
1	MMR	4	Diverse results, reduces redundancy	May miss some relevant documents
2	Similarity Threshold	2	Filters low-quality matches	Risk of excluding relevant documents
3	MultiQuery	10	Improves query coverage	Computationally expensive
4	Custom MultiQuery	13	Domain-specific query expansion	Quality depends on prompt design
5	Ensemble (BM25 + Vector)	6	Combines keyword and semantic search	More complex implementation
6	Enhanced Ensemble	14	Best of both worlds approach	Highest complexity
7	Reordered Custom MultiQuery	10	Optimizes document ordering for LLMs	Post-processing step only

Table-1: Comparison of retrievers

4.4. Justification for Selecting the Ensemble Approach

API documentation represents a domain where both precision and recall are critical:

- Users may phrase questions in natural language, requiring semantic matching to locate relevant endpoints or methods.
- Exact terms (such as endpoint paths or parameter names) must also be retrieved accurately.

A single retrieval method typically prioritizes either semantic similarity or lexical precision, but not both. The ensemble approach integrates multiple strategies to meet these competing requirements.

Components and their roles:

BM25 Retriever: Ensures retrieval of exact matches for paths and method names.

FAISS-Based Dense Retriever: Captures paraphrased or semantically related queries.

Similarity Score Threshold: Filters out low-relevance matches, improving precision.

By assigning appropriate weights, the ensemble can be tuned to optimize both recall and precision. The flexibility of the ensemble design also allows for iterative refinement based on evaluation results.

Chapter 5. QA Chain and Response Generation

5.1 Introduction

Once the retrieval component identifies relevant documentation passages, the final stage in the Retrieval-Augmented Generation (RAG) pipeline is answer generation. This stage is responsible for transforming the retrieved text into a coherent, user-friendly response that addresses the developer's question using only information present in the documentation.

To implement this, the project adopts a retrieval-augmented question-answering (QA) chain using a large language model (SLM). The approach ensures that answers are not only relevant but also properly grounded in the authoritative documentation context.

5.2 Architecture and Tooling

The QA system is implemented using the LangChain framework, which enables modular integration of retrieval and generation components. Specifically, the following components were used:

- ***SLM (Ollama, llama3.2:latest):*** A modern open-weight large language model, selected for its strong general reasoning capabilities, support for local deployment via Ollama, and controllable generation parameters (e.g. temperature).
- ***Retrievers***: An ensemble retrieval system combining semantic and keyword-based strategies, as described previously.
- ***RetrievalQA Chain***: A LangChain abstraction that couple's retrieval and generation seamlessly, enabling context-aware question answering.

5.3 QA Chain Design

The RetrievalQA chain operates in two stages:

1. Retrieval Stage: The ensemble retriever identifies a set of documentation chunks most relevant to the user query.
2. Generation Stage: The SLM receives these retrieved passages as context and produces an answer constrained to that context.

This design ensures that the generated answers remain faithful to the source documentation, mitigating the risk of hallucinated or fabricated content.

5.4 Prompt Engineering

To enforce strict grounding and maintain answer quality, a structured system prompt was designed. This system prompt instructs the SLM to:

- Use only the retrieved context to answer questions.
- Refuse to answer if the context does not contain the necessary information.
- Provide structured output, including:

- Path and method for endpoints.
- Parameters, security requirements, and schemas.
- JSON code blocks where applicable.
- Bullet points for clarity.

The system prompt includes explicit constraints such as “If the answer is not in the context, say ‘I don't know based on the provided documentation.”. Such instructions are critical in mitigating hallucination, a well-known issue in SLM-based systems, by forcing the model to anchor its answers strictly to the authoritative API documentation provided.

5.5 Context Formatting

Before generation, the retrieved documents are programmatically concatenated into a structured context block. This block is injected into the prompt alongside the user's question. The template used for this step explicitly delineates:

- The documentation context.
- The new user questions.

This design ensures the SLM receives the necessary information in a clear, machine-readable format that supports accurate context-based reasoning.

5.6 Justification for Technique

This two-stage retrieval-augmented approach was chosen over simpler question-answering systems for several reasons:

- ***Domain Complexity***: API documentation contains structured, technical information with precise semantics (e.g., HTTP methods, paths, parameter names). Retrieval-augmented methods ensure these details are available to the SLM at generation time.
- ***Robustness to User Queries***: Developers may phrase questions ambiguously or incompletely. Retrieval grounding ensures the model can handle paraphrases and incomplete questions by providing relevant documentation snippets.
- ***Accuracy***: By forcing the SLM to condition on actual documentation, the system reduces the risk of hallucinated or invented answers, which could have serious implications in an API usage context.
- ***Interpretability***: Returning the source documents alongside answers enables transparency and auditability of the system’s reasoning process.

5.7 Model Selection and Configuration

The llama3.2:latest model was selected for its balance between performance and deployability:

Supports local deployment via Ollama, ensuring data privacy and zero cost.

- Competitive performance on reasoning tasks at modest computational cost.
- Controllable generation through hyperparameters such as temperature.

A low temperature setting (e.g., 0.1) was used during answer generation to encourage deterministic, precise outputs over creative or speculative completions.

5.8 Rate Limiting and Production Considerations

To ensure stability in a production deployment, a rate limiter was integrated into the SLM interface. This mechanism prevents overloading the generation backend, ensuring consistent response times and system reliability.

5.9 Advantages, Limitations and Mitigation Strategies

- ***Faithfulness***: By tightly coupling retrieval and generation, the system guarantees that answers remain grounded in authoritative documentation.
- ***Adaptability***: The ensemble retrieval strategy ensures high recall across a range of query phrasings.
- ***Transparency***: The system can return retrieved context alongside the answer for verification.
- ***Structured Outputs***: Prompt constraints enforce clear, developer-friendly formatting (e.g., JSON code blocks).

Limitations

Context Window Limitations: The SLM can only process a limited number of tokens, which may constrain the amount of documentation included in the prompt. Mitigation strategies include chunking, relevance filtering, and document reordering.

Prompt Sensitivity: The quality of responses depends on prompt design. Extensive testing and refinement were performed to optimize instructions.

Latency: Retrieval-augmented generation can introduce additional latency, especially with large context windows or complex retrievers. Caching and rate-limiting were used to manage

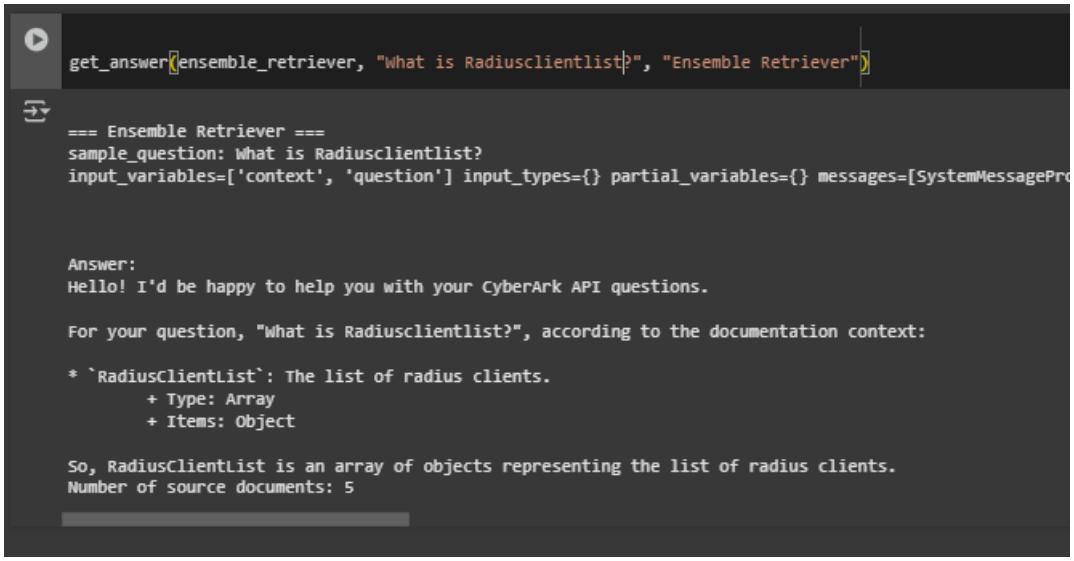


Figure-8: Sample QnA on console chat

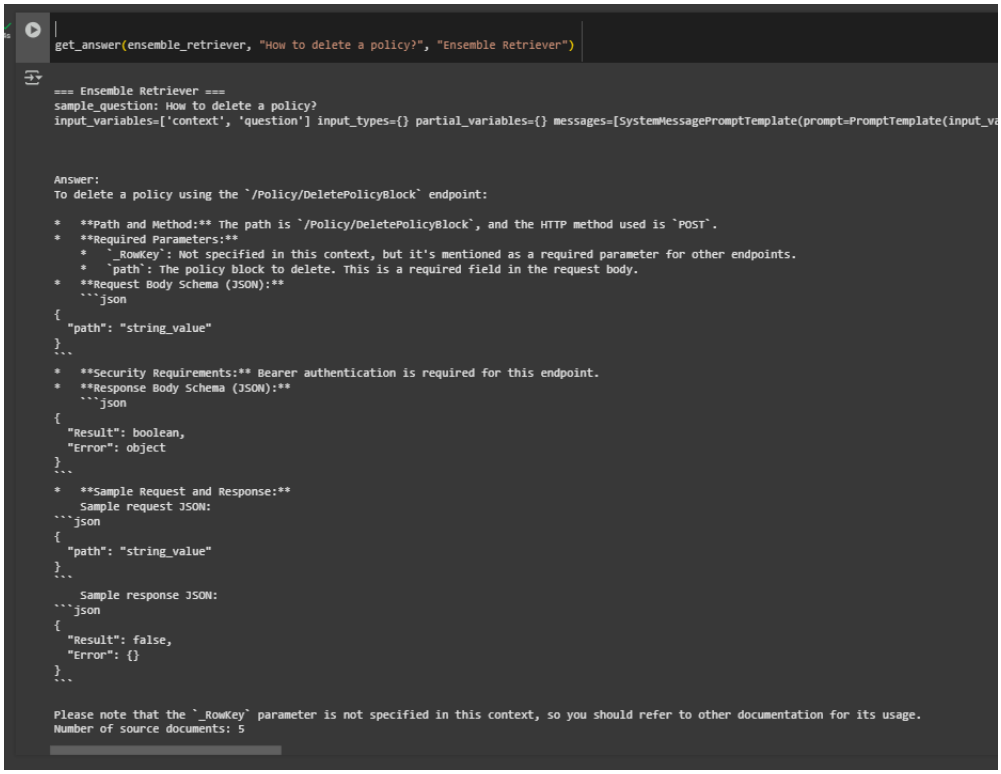


Figure-9: Sample QnA on console chat

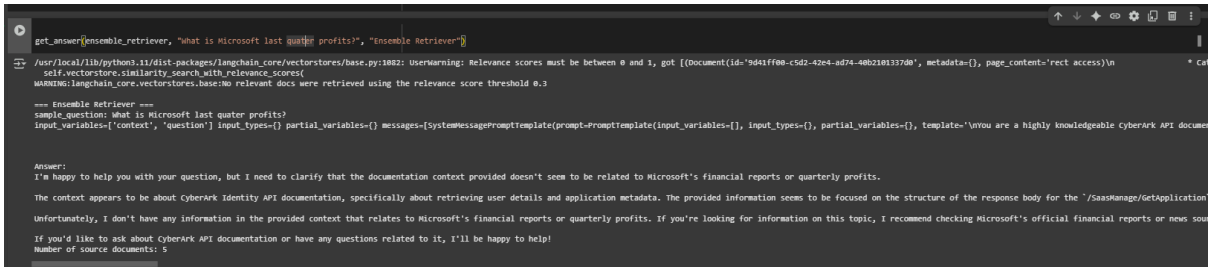


Figure-10: Sample QnA on console chat with out of context question.

Directions for Future Work

In this section, we outline several promising directions to extend and improve the Retrieval-Augmented Generation (RAG)–based chatbot developed in this project. These proposed enhancements are designed to address current limitations, improve user experience, and strengthen the system's robustness and utility in production settings.

1. Implementation of Output Parsers

One important area for further development is the introduction of output parsers tailored to the API documentation domain. While the current system generates text-based answers by retrieving and synthesizing relevant documentation chunks, it lacks fine-grained control over the output structure. Custom output parsers can enforce consistent answer formats, extract structured data such as endpoint parameters or request/response schemas, and support integration with other systems that expect machine-readable outputs. This would improve clarity and reliability for users seeking precise API details.

2. Input and Output Guardrails

Another valuable enhancement is the implementation of guardrails for both user inputs and system outputs. Input-side guardrails can detect ambiguous or malformed queries, prompting users to clarify their requests before retrieval. Output-side guardrails can filter or validate generated responses to ensure accuracy, prevent hallucinations, and enforce compliance with organizational guidelines. By introducing these safeguards, the system can reduce the risk of providing misleading or incomplete information, supporting safer and more trustworthy deployment in enterprise environments.

3. Context-Aware and Multi-Turn Dialogue

Currently, the chatbot operates primarily in a single-turn question-answering mode. Extending it to support context-aware conversations would significantly enhance usability, allowing users to ask follow-up questions and refine their queries iteratively. This requires mechanisms for dialogue state tracking and retrieval conditioning based on conversation history. Developing such capabilities would better mirror human support workflows and make the chatbot a more natural assistant for navigating complex API documentation.

4. User Interface Development

While the present implementation focuses on the backend retrieval and generation components, deploying the chatbot in real-world settings will benefit from a dedicated user interface (UI). Designing an intuitive and accessible UI will make the system more approachable to developers and other stakeholders. The interface could support chat-based interactions, query history, source document previews, and personalized settings, enhancing the overall user experience and encouraging adoption.

5. Evaluation and Benchmarking

Finally, systematic evaluation and benchmarking are critical for assessing and improving system performance. Future work should define and implement evaluation protocols that measure retrieval quality (e.g., precision, recall, relevance scoring) and generation quality (e.g., correctness, fluency, adherence to documentation). Benchmarking against existing support systems or human experts can help validate the chatbot's effectiveness and identify areas for further refinement. Regular evaluation cycles would also support iterative improvements and inform model selection and tuning decisions.

Bibliography / References

1. Web Resource
   LangChain Documentation, https://python.langchain.com/docs/versions/v0_3/
2. Web Article
   An Article on Finding the Best Open-Source Embedding Model for RAG
4. Web Article
   An Article on Running LLM Locally: A Beginner's Guide to Using Ollama
5. Research Paper
   Fei Liu, Zejun Kang, Xing Han, "Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models"
6. Web Resource
   An Article on Semantic search with FAISS
7. Research Paper
   Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, Haofen Wang "Retrieval-Augmented Generation for Large Language Models: A Survey"
8. Web Resource
   Web article on Build a Retrieval Augmented Generation (RAG) App: Part 1, Build a Retrieval Augmented Generation (RAG) App: Part 2
9. Web Resource
   A video on Learn RAG From Scratch – Python AI Tutorial from a LangChain Engineer
10. Web Resource
    CyberArk Software Ltd., *CyberArk API Documentation*, Available: https://docs.cyberark.com

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

SECOND SEMESTER 2024-25

DSECLZG628T / **AIMLCZG628T DISSERTATION**

Dissertation Title: Predictive Machine Learning System for CyberArk Product Sales

Name of Supervisor: Sasikiran Varikunta

Name of Student: Saya Venkata Thrivedi

ID No. of Student: 2023AA05203

Checklist:

Is the Cover page in proper format?	Y
Is Abstract included in the Report? Is it properly written?	Y
Does the Table of Contents page include chapter page numbers	Y
Are the Pages numbered properly?	Y
Are the Figures numbered properly?	Y
Are the Tables numbered properly?	Y
Are the Captions for the Figures and Tables proper?	Y
Does the Report have Future Work / Recommendations of the work?	Y
Are References/Bibliography given in the Report?	Y
Have the References been cited in the Report?	Y
Is the citation of References / Bibliography in proper format?	Y

Student:
Saya Venkata Thrivedi

Supervisor:
Sasikiran Varikunta